CS2108
AY24/25 SEM 2
github/Gavino3o

Sinusoids

- Sinusoids are periodic functions that can be represented as a sum of sine and cosine functions. The general form of a sinusoid is:

$$y(t) = A\sin(\omega t + \phi) + B\cos(\omega t + \phi) \quad (1)$$

where:
- A is the amplitude
- ω is the angular frequency
- ϕ is the phase shift
- B is the vertical shift

- The angular frequency ω is related to the frequency f and the period T by:

$$\omega = 2\pi f = 2\pi\frac{1}{T} \quad (2)$$

- Hence, the sinusoid can also be expressed in terms of frequency:

$$y(t) = A\sin(2\pi f t + \phi) + B\cos(2\pi f t + \phi) \quad (3)$$

Discrete Time Sinusoids

- Discrete time sinusoids are sampled versions of continuous time sinusoids. The general form is:

$$x[n] = \sum_{k=0}^{K-1} a_k \sin(2\pi f_k t_n) + b_k \cos(2\pi f_k t_n) \quad (4)$$

where:
- n is the discrete time index
- k is the frequency index
- t_n is the discrete time variable $n\Delta t$
- f_k is the frequency of the k-th component $k\Delta f$
- Δt is the sampling period
- Δf is the frequency resolution
- K is the total number of frequencies

The coefficients a_k and b_k define the amplitude and phase of each frequency component.

- The discrete time sinusoid can also be expressed in different forms:

$$\begin{aligned} x[n] &= \sum_{k=0}^{K-1} a_k \sin(2\pi f_k t_n) + b_k \cos(2\pi f_k t_n) \\ &= \sum_{k=0}^{K-1} c_k \sin(2\pi f_k t_n + \phi_k) \\ &= \sum_{k=0}^{K-1} d_k \cos(2\pi f_k t_n + \theta_k) \end{aligned} \quad (5)$$

where:
- Second line is the same sinusoid a summation of shifted sine functions
- Third line is the same sinusoid as a summation of shifted cosine functions
- c_k and d_k are the coefficients for the sine and cosine forms
- ϕ_k and θ_k are the phase shifts for the sine and cosine forms
- The relationship between the coefficients is given by:

$$\begin{aligned} c_k &= \sqrt{a_k^2 + b_k^2} \\ d_k &= \sqrt{a_k^2 + b_k^2} \\ \phi_k &= \tan^{-1}\left(\frac{b_k}{a_k}\right) \\ \theta_k &= \tan^{-1}\left(\frac{a_k}{b_k}\right) \end{aligned} \quad (6)$$

- If $a_k = b_k$, the sinusoid is a combination of sine and cosine functions with equal amplitude and phase shift. Phase shift is 45 degrees.
- If a_k = -b_k, the sinusoid is a combination of sine and cosine functions with equal amplitude and phase shift. Phase shift is 135 degrees.
- If $a_k = 0$, phase shift is 90 degrees
- If $b_k = 0$, phase shift is 0 degrees

Periodicity

- Fundamental period is the smallest period of a sinusoid. The fundamental period is the time it takes for the sinusoid to complete one full cycle.
- The fundamental frequency is the reciprocal of the fundamental period:

$$f_0 = \frac{1}{T_0} \quad (7)$$

- The fundamental frequency is the lowest frequency of a sinusoid. It is the frequency at which the sinusoid oscillates.
- Sinusoids are periodic functions, meaning they repeat their values at regular intervals.
- If we add sinoiuds with different frequencies, the resulting function is not periodic unless the frequencies are rationally related.
- Suppose we have a sinusoid with the equation:

$$y(t) = a_1\sin(\omega_1 t + \phi_1) + a_2\cos(\omega_2 t + \phi_2) \quad (8)$$

- To determine if the sinusoid is periodic, we need to check if the ratio of the **frequencies** is rational:

$$\frac{\omega_1}{\omega_2} = \frac{p}{q} \quad (9)$$

where p and q are integers.
- To determine the **fundamental period** of the sinusoid, we need to find the least common multiple (LCM) of the periods of the individual sinusoids:

$$T = \text{lcm}(T_1, T_2) \quad (10)$$

where T_1 and T_2 are the periods of the individual sinusoids. Calculate the period of each sinusoid using:

$$T_i = \frac{2\pi}{\omega_i} \quad (11)$$

- Hint: Let ratio of periods = $\frac{T_1}{T_2} = \frac{q}{p}$, then $T = pT_1 = qT_2$
- Suppose we have a sinusoid with the equation:

$$y(t) = a_1\sin(2\pi f_1 t + \phi_1) + a_2\cos(2\pi f_2 t + \phi_2) \quad (12)$$

where f_1 and f_2 are integers
- To determine the **fundamental frequency** of the sinusoids, we need to find the greatest common divisor (GCD) of the frequencies of the individual sinusoids:

$$f = \text{gcd}(f_1, f_2) \quad (13)$$

- Then fundamental period is: $T = \frac{1}{f}$
- The above calculatiosn can be done with sinusoids of more than 2 frequencies as well.

Fourier Series

Recap

- **Orthogonal Basis Set** a set of functions that are orthogonal to each other. This means that the inner product of any two functions in the set is zero.
- Inner product of two functions $f(t)$ and $g(t)$ is defined as:

$$\langle f(t), g(t)\rangle = \int_{-\infty}^{\infty} f(t)g(t)dt \quad (14)$$

- Inner Product (continuous) = Dot Product (discrete)
- **Sine and cosine functions are orthogonal to each other**, no matter their frequencies. Inner/Dot product = **zero**.

Let $f_m = mf_0$ and $f_n = nf_0$ where m and n are integers

$$\int_0^T \sin(2\pi f_m t)\cos(2\pi f_n t)\,dt = \frac{1}{2}\left[\frac{\cos(2\pi(f_m - f_n)t)}{-2\pi(f_m - f_n)} - \frac{\cos(2\pi(f_m + f_n)t)}{2\pi(f_m + f_n)}\right]_0^T$$

$$= \begin{cases} 0 & if\ m \neq n \\ 0 & if\ m == n \end{cases}$$

- **Sine waves of different frequencies** are orthogonal to each other.Inner/Dot product = **zero**.

Let $f_m = mf_0$ and $f_n = nf_0$ where m and n are integers

$$\int_0^T \sin(2\pi f_m t)\sin(2\pi f_n t)\,dt = \frac{1}{2}\left[\frac{\sin(2\pi(f_m - f_n)t)}{2\pi(f_m - f_n)} - \frac{\sin(2\pi(f_m + f_n)t)}{2\pi(f_m + f_n)}\right]_0^T$$

$$= \begin{cases} 0 & if\ m \neq n \\ \frac{T}{2} & if\ m == n \end{cases}$$

- Similarly, **Cosine waves of different frequencies** are orthogonal to each other. Inner/Dot product = **zero**.

Let $f_m = mf_0$ and $f_n = nf_0$ where m and n are integers

$$\int_0^T \cos(2\pi f_m t)\cos(2\pi f_n t)\,dt = \frac{1}{2}\left[\frac{\sin(2\pi(f_m - f_n)t)}{2\pi(f_m - f_n)} + \frac{\sin(2\pi(f_m + f_n)t)}{2\pi(f_m + f_n)}\right]_0^T$$

$$= \begin{cases} 0 & if\ m \neq n \\ \frac{T}{2} & if\ m == n \end{cases}$$

- For sine and cosine with non-zero phase shifts:

$$\begin{aligned} \sin(2\pi f t + \phi) &= sin(2\pi f t)\cos(\phi) + \cos(2\pi f t)\sin(\phi) \\ \cos(2\pi f t + \phi) &= cos(2\pi f t)\cos(\phi) - \sin(2\pi f t)\sin(\phi) \end{aligned} \quad (15)$$

- In other words, with a phase shift, a sine or cosine function that is phase shifted can be expressed as a weighted sum of sine and cosine functions.
- So in general, the sum of sinusoidal signals, even with non-zero (but constant) phase, can be written as a linear combination of sine and cosine with zero phase.

$$\sum_{k=0}^{K-1} A_k \sin(2\pi f_k t + \phi_k) = \sum_{k=0}^{K-1} A_k\,[\sin(2\pi f_k t)\cos(\phi_k) + \cos(2\pi f_k t)\sin(\phi_k)]$$

$$\sum_{k=0}^{K-1} B_k \cos(2\pi f_k t + \phi_k) = \sum_{k=0}^{K-1} A_k\,[\cos(2\pi f_k t)\cos(\phi_k) - \sin(2\pi f_k t)\sin(\phi_k)]$$

Fourier Series

- Fourier series is a way to represent a periodic signal as a sum of sine and cosine functions. It is used to analyze periodic signals in the frequency domain.
- **Sine and cosine functions make up the basis functions**. I.e for a sinusoidal signal with highest frequency f_m, we can represent it as a sum of M sine and cosine functions with frequencies $f_0, f_1, \ldots, f_{M-1}$.

$$y(t) = \sum_{m=0}^{M-1} (a_m \sin(2\pi f_m t) + b_m \cos(2\pi f_m t)) \quad (16)$$

- How do we determine all of the a_m and b_m:

Recall that we can represent any periodic signal by a linear combination of shifted sines, i.e.

$$\begin{aligned} any.periodic.signal &= \sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m) \\ &= \sum_{m=0}^{M-1} A_m\,(\sin(2\pi f_m t)\cos(\phi_m) + \cos(2\pi f_m t)\sin(\phi_m)) \\ &= \sum_{m=0}^{M-1} [\,A_m\cos(\phi_m)\sin(2\pi f_m t) + A_m\sin(\phi_m)\cos(2\pi f_m t)\,(\ \ ) \\ &= \sum_{m=0}^{M-1} [\,\underbrace{A_m\cos(\phi_m)}_{a_m}\sin(2\pi f_m t) + \underbrace{A_m\sin(\phi_m)}_{b_m}\cos(2\pi f_m t)\,] \end{aligned}$$

$$\begin{aligned} a_m &= A_m\cos(\phi_m) \\ b_m &= A_m\sin(\phi_m) \end{aligned} \quad (17)$$

- The Fourier series coefficients a_m and b_m are calculated using the following formulas:

$$\begin{aligned} a_m &= \frac{2}{T}\int_0^T y(t)\sin(2\pi f_m t)dt \\ b_m &= \frac{2}{T}\int_0^T y(t)\cos(2\pi f_m t)dt \end{aligned} \quad (18)$$

Recall the following integration results [Dot Product]

Mixed Signal

$$\int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)]\sin(2\pi f_k t)\,dt = \frac{T}{2}A_k\cos(\phi_k)$$

$$\int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)]\cos(2\pi f_k t)\,dt = \frac{T}{2}A_k\sin(\phi_k)$$

$$\int_0^T [\sum_{m=0}^{M-1} B_m \cos(2\pi f_m t + \phi_m)]\sin(2\pi f_k t)\,dt = \frac{-T}{2}A_k\sin(\phi_k)$$

$$\int_0^T [\sum_{m=0}^{M-1} B_m \cos(2\pi f_m t + \phi_m)]\cos(2\pi f_k t)\,dt = \frac{T}{2}A_k\cos(\phi_k)$$

We can therefore use $\sin(2\pi f_k t)$ and $\cos(2\pi f_k t)$ to figure out what are all the a_m and b_m in the signal.

$$\int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)] \sin(2\pi f_k t)\, dt = \frac{T}{2} A_k \cos(\phi_k) = \frac{T}{2} a_k$$

$$\int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)] \cos(2\pi f_k t)\, dt = \frac{T}{2} A_k \sin(\phi_k) = \frac{T}{2} b_k$$

- The ratio of b_k to a_k gives the phase shift of the sinusoid:

$$\frac{b_k}{a_k} = \frac{A_k \sin(\phi_k)}{A_k \cos(\phi_k)} = tan(\phi_k)$$
$$\phi_k = \tan^{-1}\left(\frac{b_k}{a_k}\right) \quad (19)$$

- We can use complex numbers to represent the Fourier series:

$$X(f_k) = \int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)] \cos(2\pi f_k t)\, dt$$

$$-j \int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m)] \sin(2\pi f_k t)\, dt$$

$$= \frac{T}{2} A_k \sin(\phi_k) \; - j \frac{T}{2} A_k \cos(\phi_k)$$

$$= \frac{T}{2} (\, b_k \, - \, j \, a_k \,)$$

- **Magnitude** = $|X_k| = \frac{T}{2}\sqrt{a_k^2 + b_k^2}$
- **Phase** = $\tan^{-1}\left(\frac{b_k}{a_k}\right)$
- We can also express the Fourier series in terms of **complex exponentials**, recall Eulers formula: $e^{j\theta} = \cos(\theta) + j\sin(\theta)$, then:

$$X(f_k) = \int_0^T [\sum_{m=0}^{M-1} A_m \sin(2\pi f_m t + \phi_m) e^{-j2\pi f_k t} dt \quad (20)$$

- **Fourier series formula** (important):

$$X(f_k) = \int_0^T x(t) e^{-j2\pi f_k t} dt \quad (21)$$

Fourier Transform

- Fourier transform is a way to represent a non-periodic signal as a sum of sine and cosine functions. It is used to analyze non-periodic signals in the frequency domain.
- **Intuition:** For non-periodic signals, we can think that its *period is infinite*. Then the whole signal can be imagined as periodic with infinite period.
- Fourier series comparison:

$$X(f) = \int_0^T x(t) e^{-j2\pi f t} dt \quad (22)$$

- Fourier transform is defined as:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi f t} dt \quad (23)$$

- The inverse Fourier transform is defined as:

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi f t} df \quad (24)$$

- The Fourier transform can be used to analyze the frequency content of a signal. It can be used to determine the amplitude and phase of each frequency component in the signal.
- Fourier transform equation using radian frequency ω:

$$X(\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt \quad (25)$$

- The inverse Fourier transform using radian frequency ω:

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\omega) e^{j\omega t} d\omega \quad (26)$$

- The Fourier transform can also be used to filter signals. By multiplying the Fourier transform of a signal by a filter function, we can remove or enhance certain frequency components in the signal.

Discrete Fourier Transform (DFT)

- **DFT** Formula:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi k \frac{n}{N}} \quad (27)$$

- **Inverse DFT** Formula:

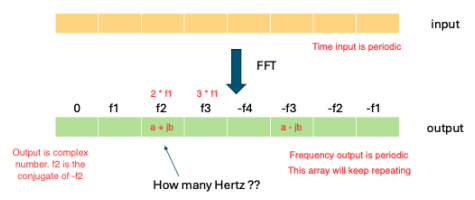
$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi k \frac{n}{N}} \quad (28)$$

where:
- N is the number of samples
- k is the frequency index (discretised frequencies)
- n is the time index
- $x[n]$ is the input signal
- $X[k]$ is the DFT of the input signal

- FFT (Fast Fourier Transform) is an efficient algorithm to compute the DFT. It reduces the computational complexity from $O(N^2)$ to $O(N \log N)$.

Frequency Bins

- Frequency resolution: Smallest frequency difference that can be resolved by the DFT. It is determined by the sampling frequency and the number of samples.
- The DFT maps the input signal to a set of frequency bins. Each bin corresponds to a specific frequency range.



- The frequency resolution of the DFT is given by:

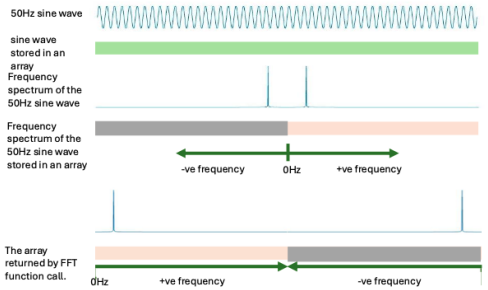
$$\Delta f = \frac{f_s}{N} \quad (29)$$

where:
- f_s is the sampling frequency
- N is the number of samples

- The frequency bins are given by:

$$f_k = k\Delta f \quad (30)$$

- The DC component (k=0) corresponds to the average value of the signal. It is a constant in the sinusoid equation. The Nyquist frequency (k=N/2) corresponds to the highest frequency that can be represented by the DFT.
- We can imagine the DC component as vertical offset of the signal from the x-axis.
- **Visualisation**:



- Derivation of the frequency bins:

$$H(f) = \int_{t=-\infty}^{\infty} h(t)\, e^{-j2\pi f t}\, dt$$

$$H(f) \approx \sum_{n=0}^{N-1} h[n]\, e^{-j2\pi f n \Delta t}\, \Delta t$$

$$H(f) \approx \Delta t \sum_{n=0}^{N-1} h[n]\, e^{-j2\pi f n \Delta t}$$

$$H(f) \approx \frac{1}{f_s} \sum_{n=0}^{N-1} h[n]\, e^{-j2\pi \frac{f}{f_s} n}$$

Let $f = k\,\Delta f$, i.e. we also discretize the frequency axis using a frequency step Δf

$$H[k] \approx \frac{1}{f_s} \sum_{n=0}^{N-1} h[n]\, e^{-j2\pi \frac{k\Delta f}{f_s} n}$$

Recall that $H[k] \approx \sum_{n=0}^{N-1} h[n]\, e^{-j2\pi \frac{k}{N} n}$

Ignoring the scale factor, we have

$$2\pi \frac{k\Delta f}{f_s} n = 2\pi \frac{k}{N} n$$

Therefore,

$$\frac{\Delta f}{f_s} = \frac{1}{N}$$

- **Note**: Sometimes, we pad the input signal with zeros to increase the number of samples. Padding zeros to the input decreases the frequency bin size, but does not increase the frequency resolution.
- **Note**: N = 2^n for FFT, where n is an integer. This is because the FFT algorithm is based on the divide-and-conquer approach, which works best when the number of samples is a power of 2.

Properties of DFT

- **Intuition**: When you discretise one domain, you also discretise the other domain. The *other domain becomes periodic*.
- **Linearity**: $\mathcal{F}\{ax[n] + by[n]\} = aX[k] + bY[k]$
- **Time Shifting**: $\mathcal{F}\{x[n - n_0]\} = X[k]e^{-j2\pi k \frac{n_0}{N}}$
- If you shift a signal $x[n]$ by m samples in time, the Fourier transform $X[k]$ remains the same in magnitude, but is multiplied by a complex exponential.
- This complex exponential introduces a linear phase shift in the frequency domain. This exponential has a magnitude of 1, so the magnitude spectrum stays unchanged.
- **Frequency Shifting**: $\mathcal{F}\{x[n]e^{j2\pi m \frac{n}{N}}\} = X[k - m]$
- If you multiply $x[n]$ by a complex exponential in the time domain, it causes a circular shift in the frequency domain. Shifting the frequency response by m bins. Both magnitude and phase are affected.
- **Time Reversal**: $\mathcal{F}\{x[-n]\} = X^*[k]$
- **Conjugate Symmetry**: $X[k] = X^*[N - k]$
- Here $X^*[k]$ is the complex conjugate of $X[k]$. (i.e If $X[k] = a + jb$, then $X^*[k] = a - jb$)
- It is important to note that the $|X[k]| = |X^*[k]|$

- Conversion **from continuous to discrete**:

Let $t = n\,.\Delta t = n\,\frac{T}{N}$ where N is the total number of points

$-\infty < t < \infty \quad \rightarrow \quad -\infty < n < \infty$

$$X(\omega) = \int_{-\infty}^{\infty} x(t)\, e^{-j\omega t}\, dt \quad \rightarrow \quad X(\omega) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-j\omega n.\Delta t}\, \Delta t$$

$$= \frac{T}{N} \sum_{n=-\infty}^{\infty} x[n]\, e^{-j\omega \frac{nT}{N}}$$

- **Note**: $\Delta t = \frac{T}{N}$

Let $\omega = k\,\omega_0 = k\,2\pi f_0 = k\,2\pi \frac{1}{T}$

$$X[k] = \frac{T}{N} \sum_{n=-\infty}^{\infty} x[n]\, e^{-jk2\pi \frac{1}{T} \frac{nT}{N}}$$

$$X[k] = \frac{T}{N} \sum_{n=-\infty}^{\infty} x[n]\, e^{-j2\pi k \frac{n}{N}}$$

- DFT output is **periodic with period N** (repeats every N samples). $X[k] = X[k + N]$

$$X[k] = \frac{T}{N}\sum_{n=-\infty}^{\infty} x[n]\ e^{-j2\pi k\frac{n}{N}}$$

$$X[k+N] = \frac{T}{N}\sum_{n=-\infty}^{\infty} x[n]\ e^{-j2\pi(k+N)\frac{n}{N}}$$

$$= \frac{T}{N}\sum_{n=-\infty}^{\infty} x[n]\ e^{-j2\pi k\frac{n}{N} - j2\pi n}$$

e^{-j*2*pi*n} = cos(2*pi*n) - j sin(2*pi*n) = 1

$$= \frac{T}{N}\sum_{n=-\infty}^{\infty} x[n]\ e^{-j2\pi k\frac{n}{N}}$$

$$= X[k]$$ → $X[k]$ is periodic with period N

- Similarly, the input signal is periodic with period N. $x[n] = x[n+N]$

$$x[n] = \frac{1}{N}\sum_{k=-\infty}^{\infty} X[k]e^{j2\pi k\frac{n}{N}} \quad (31)$$

- **Summary:**

continuous time, continuous frequency

$$X(\omega) = \int_{-\infty}^{\infty} x(t)\ e^{-j\omega t}\,dt$$ CTFT — Continuous Time Fourier Transform

discrete time, continuous frequency (i.e. ω is continuous)

Used in Sampling

$$X(\omega) = \frac{T}{N}\sum_{n=-\infty}^{\infty} x[n]\ e^{-j\omega\frac{nT}{N}}$$ DTFT — Discrete Time Fourier Transform. Time axis is discretized but frequency axis is not

discrete time, discrete frequency (i.e. k is discrete)

NOTE: the N here is not the period in seconds. The N here is an integer representing the total number of sampled points within one period.

$$X[k] = \sum_{n=0}^{N-1} x[n]\ e^{-j2\pi k\frac{n}{N}}$$ DFT (FFT) — Discrete Fourier Transform (DFT) (Fast Fourier Transform is a fast implementation of DFT)

A/Prof NG Teck Khim

continuous time, continuous frequency

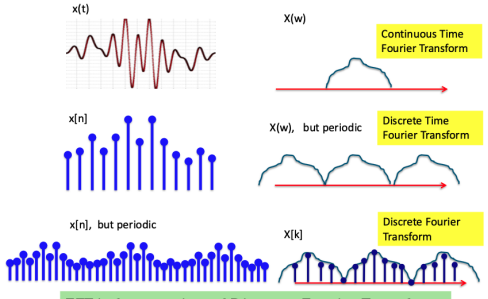
$$x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(\omega)\ e^{j\omega t}\,d\omega$$ Inverse CTFT

discrete time, continuous frequency (i.e. k is continuous)

$$x[n] = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(\omega)\ e^{j\omega\frac{nT}{N}}\,d\omega$$ Inverse DTFT

discrete time, discrete frequency (i.e. k is discrete)

$$x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]\ e^{j2\pi k\frac{n}{N}}$$ Inverse DFT(iFFT)



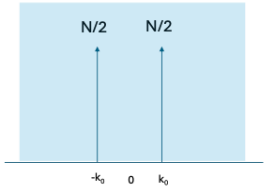
FFT is fast version of Discrete Fourier Transform

Useful Fourier Transform Results

- Fourier transform of an **impulse** in the time domain will result in a constant function in the frequency domain. [1, 0, 0, 0] → [1, 1, 1, 1]
- Fourier transform of a **constant function** in the time domain will result in an impulse in the frequency domain. [1, 1, 1, 1] → [4, 0, 0, 0]

Discrete Fourier Transform Pairs

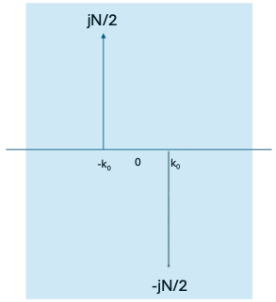
$$\cos(\frac{2\pi}{N}k_0 n) \underset{\text{iFFT}}{\overset{\text{FFT}}{\rightleftarrows}} \frac{N}{2}(\delta[k-k_o] + \delta[k+k_o])$$



Only real component
Amplitude of N/2
Fundamental Frequency, will have a spike

Discrete Fourier Transform Pairs

$$\sin(\frac{2\pi}{N}k_0 n) \underset{\text{iFFT}}{\overset{\text{FFT}}{\rightleftarrows}} \frac{-jN}{2}\delta[k-k_o] + \frac{jN}{2}\delta[k+k_o]$$



Only imaginary component

Continuous Time Fourier Transform for an Impulse Train

$$p(t) = \sum_{n=-\infty}^{\infty}\delta(t-nT_s) \longleftrightarrow P(\omega) = \sum_{k=-\infty}^{\infty}\omega_0\,\delta(\omega-k\omega_0)$$

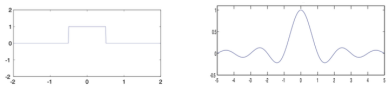
where $\omega_0 = \frac{2\pi}{T_s}$

Continuous Time Fourier Transform for a Rectangular Function

$$rect(t) = \begin{cases} 1 & -0.5 \le t \le 0.5 \\ 0 & otherwise \end{cases}$$

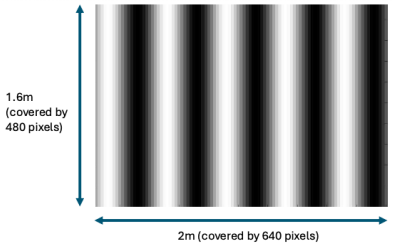
L'Hopital's Rule

$$x(t) = rect(t) \longleftrightarrow X(\omega) = \frac{2\sin(\frac{\omega}{2})}{\omega} = sinc(\frac{\omega}{2\pi})$$



2D Fourier Transform

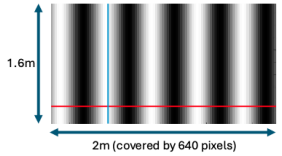
- The 2D Fourier transform is used to analyse 2D signals, such as images. Units: **cycles per meter**.
- Sampling frequency: f_s = Number of points/pixels, N per metre
- Frequency bin size: $\Delta f = \frac{f_s}{N}$



Suppose this scene measuring 1.6m by 2m is imaged by a camera of 480 pixels by 640 pixels. pixel -> a point in the wave

In the horizontal direction, the sampling frequency f_s is 640/2 = 320 samples per metre

In the vertical direction, the sampling frequency f_s is 480/1.6 = 300 samples per metre



Sample 640 along red line and perform FFT.
Horizontal frequency bin size is given by

$$horizontal\ \ f_{res} = \frac{f_s}{640} = \frac{320}{640} = 0.5\ Hz$$

Sample 480 along orange line and perform FFT.
Vertical frequency bin size is given by

$$vertical\ \ f_{res} = \frac{f_s}{480} = \frac{300}{480} = 0.625\ Hz$$

What about frequencies along other lines and frequencies along directions not horizontal or vertical ?
We will need 2D Fourier Transform (2D FFT).

- **Intuition:** Applying 1D Fourier transform to each row of the image, then applying 1D Fourier transform to each column of the result.
- 2D Fourier transform (spatial domain to frequency domain):

$$G[k,l] = \sum_{u=0}^{M-1}\sum_{v=0}^{N-1} g[u,v]e^{-j2\pi(\frac{ku}{M}+\frac{lv}{N})} \quad (32)$$

- 2D inverse Fourier transform (frequency domain to spatial domain):

$$g[u,v] = \frac{1}{MN}\sum_{k=0}^{M-1}\sum_{l=0}^{N-1} G[k,l]\cdot e^{j2\pi(\frac{ku}{M}+\frac{lv}{N})} \quad (33)$$

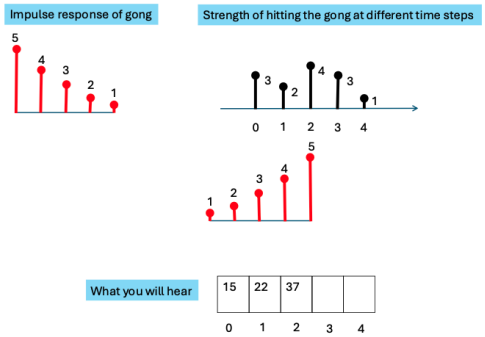
where:
  - $g[u,v]$ is the input image
  - $G[k,l]$ is the 2D Fourier transform of the input image
  - M and N are the dimensions of the image (horizontal and vertical respectively)
  - k and l are the frequency indices
  - u and v are the spatial indices
- For 2D DFT results with DC component at the top left corner, the top row represents horizontal frequencies, and the left column represents vertical frequencies.
- For 2D DFT results, the output is **periodic in both dimensions**. The output is a 2D array of complex numbers, where each element represents the amplitude and phase of a specific frequency component in the image.
- **Frequency at peak** = (Coordinate at peak - Coordinate of DC component) × Frequency bin size

Convolution

- Convolution is a mathematical operation that combines two functions to produce a third function.
- Convolution is used to filter signals and images. It is used to apply a filter to a signal or image to enhance or suppress certain features.

1D Convolution

- Process of sliding a window (kernel) over a signal, and computing the weighted sum of the signal values within the window.
- The kernel defines the weights, and it is the **flipped** version of impulse response.

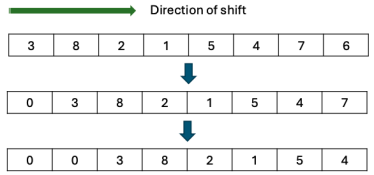


- Think of the hitting pattern as the signal, and the impulse response as the kernel.

Linear Convolution

Linear Convolution

- What we had done so far is linear convolution
- In linear convolution, the values in an array gets "shifted out"



- Linear convolution can be done in the frequency domain using the Fourier Transform.
- **Linear shift invariance property**: The output of a linear time-invariant system is the convolution of the input signal with the impulse response of the system.
- The convolution theorem states that the **Fourier transform of the convolution of two signals is equal to the product of their Fourier transforms**:

$$\mathcal{F}\{x[n] * h[n]\} = X[k]H[k] \quad (34)$$

- Derivation of the convolution theorem (discrete version):

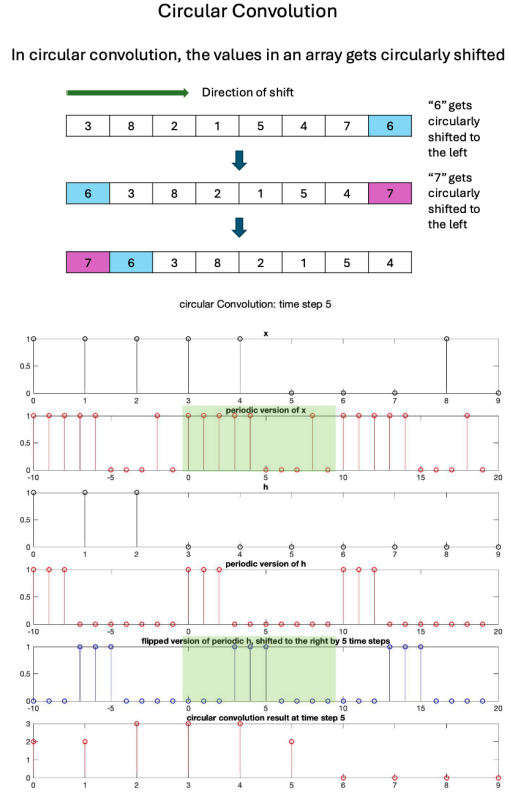
$$
\begin{aligned}
G[k] &= \mathcal{F}\{g[m]\} = \mathcal{F}\{x[m] * h[m]\} \\
&= \sum_{m=0}^{M-1} \left(\sum_{\tau=0}^{M-1} x[\tau] h[m-\tau] \right) e^{-j2\pi k \frac{m}{M}} \\
&= \sum_{\tau=0}^{M-1} x[\tau] \left(\sum_{m=0}^{M-1} h[m-\tau] e^{-j2\pi k \frac{m}{M}} \right) \\
&= \sum_{\tau=0}^{M-1} x[\tau] H[k] e^{-j2\pi k \frac{\tau}{M}} \\
&= H[k] \sum_{\tau=0}^{M-1} x[\tau] e^{-j2\pi k \frac{\tau}{M}} \\
&= H[k] X[k]
\end{aligned}
\tag{35}
$$

- This uses the circular shift property of DFT, where:

$$\mathcal{F}\{h[n-m]\} = H[k] e^{-j2\pi k \frac{m}{N}} \tag{36}$$

- Note: `ifft(fft(h(t)) .* fft(x(t)))`
- **Length of the output signal** is $N + M - 1$, where N is the length of the input signal and M is the length of the kernel.

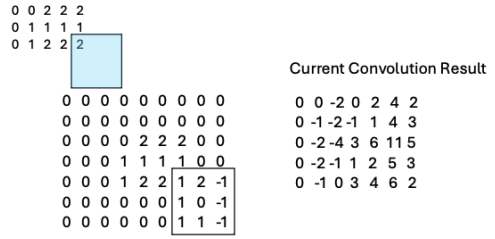
Circular Convolution



- Circular convolution is a special case of linear convolution, where the input signal and kernel are both periodic.
- To achieve circular convolution, we need to ensure that the input signal and kernel are both periodic with the same period. This is done by **zero-padding** the input signal and kernel to the same length.
- Normally circular convolution is "unwanted" because it can cause aliasing and distortion in the output signal.
- To prevent circular convolution, we can use zero-padding by adding zeros to the input signal and kernel before performing the convolution.
- We pad $x[n]$ with $M-1$ zeros, and $h[n]$ with $N-1$ zeros. This ensures that the output signal has a length of $N+M-1$.

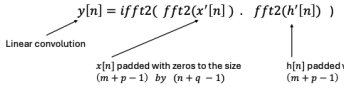
2D Convolution

- 2D convolution is used to filter images. It is similar to 1D convolution, but it uses a 2D kernel (filter) that slides over the image.
- The input image is treated as a 2D array with zeros padded around the edges.
- The kernel is flipped in both dimensions before convolution. (left-right and up-down)
- The kernel is applied to each pixel in the image, and the output pixel is computed as the weighted sum of the pixels in the kernel.
- Start from the top left corner of the image, and slide the kernel over the image.



- To implement linear convolution using FFT, we will need to handle the periodicity assumption of the DFT. This is again done by **zero-padding the input image and kernel**.

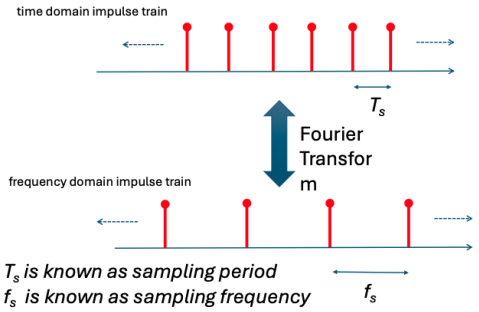
In general, if image $x[n]$ is of size m by n, and the kernel $h[n]$ is of size p by q, then both the image and kernel must be padded with zeros to the minimum size of $(m+p-1)$ by $(n+q-1)$ before performing the 2D FFT.



- For better perforamnce, we can **pad the input image and kernel to the next power of 2**. This is because the FFT algorithm is based on the divide-and-conquer approach, which works best when the number of samples is a power of 2.

Sampling Theorem

- Fun fact: 44100 Hz is the standard sampling frequency for audio signals. This is because it is twice the highest frequency that can be heard by humans, which is 20 kHz.
- Recall that the the Fourier transform of an impulse train is another impulse train.



- **Multiplication in one domain corresponds to convolution in the other domain.**

Convolution with Impulse and Train of Impulses

Convolution in frequency domain:

$$G(f) = H(f) * X(f) = \int_{-\infty}^{\infty} H(\tau)\, X(f-\tau)\, d\tau$$

Convolving an impulse at nf_s with $X(f)$ gives

Integration of impulse (Area under the curve) = 1

Train of impulses

$$G(f) = \sum_{n=-\infty}^{\infty} [\, \delta(f - nf_s) * X(f)\,] = \sum_{n=-\infty}^{\infty} [\, \int_{-\infty}^{\infty} \delta(\tau - nf_s)\, X(f-\tau)\, d\tau\,]$$

$$G(f) = \sum_{n=-\infty}^{\infty} X(f - nf_s)$$

Replication of $X(f)$ at nf_s

- **Convolution with a delta function shifts the signal in time.**

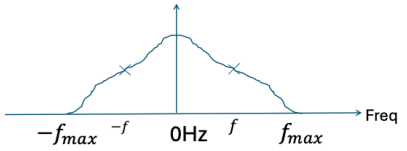
$$\delta(f-a) * X(f) = X(f-a) \tag{37}$$

- Sampling of a signal is represented as multiplcation of the signal (time domain) with an impulse train (time domain).
- In the **frequency domain**, this becomes convolution with an impulse train.
- This results in a replication of spectrum $X(f)$ at the integer multiples of the sampling frequency f_s.

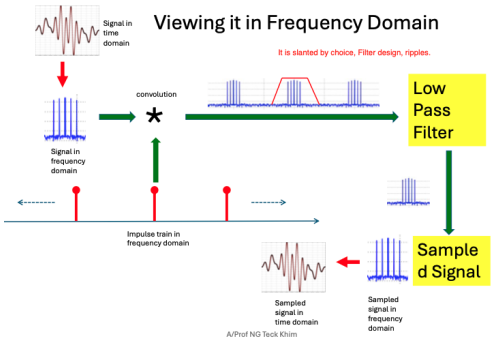
Note that for real signal, its Fourier Transform exhibits conjugate symmetry, i.e.

$$X(f) = X^*(-f)$$

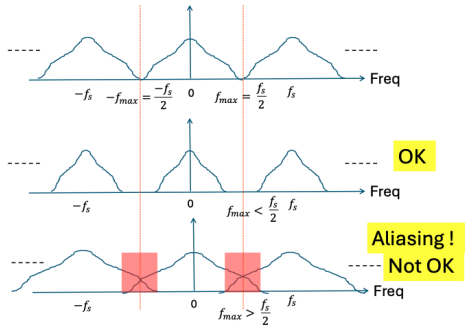
So the magnitude of Fourier Transform for real signal is always symmetrical about the y-axis (i.e. the zero-frequency line)



Note that in DFT, it is also true that $X[k] = X^*[-k]$ if the signal is real in the time domain.



- **Aliasing**:



- **Nyquist Rate**: The minimum sampling frequency required to avoid aliasing is twice the highest frequency f_{max} present in the signal.

$$f_s > 2f_{max} \tag{38}$$

- Given the maximum sampling frequency available f_s, we need to make sure the signal does not contain frequency components above $\frac{1}{2} f_s$.
- This can be done by using a **low-pass filter** before sampling the signal. The low-pass filter will remove the high-frequency components from the signal.

Data Compression

- Lossy: JPEG, MP3, MPEG
- Lossless: PNG, LZW used in GIF, FLAC

Lossy Compression

- Sacrifice part of the signal lease critical to human perception.
- Image: Missing crispness, color, and detail.
- Audio: Missing high pitch sounds, and low volume sounds.
- **Strategy:** Remove high frequency components.

Discrete Cosine Transform (DCT)

- Used in JPEG, MPEG, and MP3 compression.
- Similar to Fourier transform, but uses cosine functions instead of sine and cosine functions.
- DCT itself is not lossy, it is the filtering of the DCT coefficients that is lossy.
- DCT output is real-valued, while DFT output is complex-valued.

- **Recap:** Cosine is an even function, and sine is an odd function. Hence, Fourier transform of cosine is real, and Fourier transform of sine is imaginary.
- **Even function** exhibits symmetry about the y-axis, $x(t) = x(-t)$
  Odd function exhibits symmetry about the origin (anti-symmetry) $x(t) = -x(-t)$.
- Hence, even signals can be fully represented by cosine functions, and odd signals can be fully represented by sine functions.
- Fourier transform of an **even signal is real** (cosine basis functions only), and Fourier transform of an **odd signal is imaginary** (sine basis functions only).
- There are different types of DCT, but the most common one is **DCT-II**, which is used in JPEG compression.

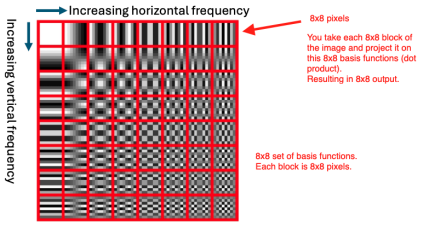
$$X[k] = \sum_{n=0}^{N-1} x[n] \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right) = \sum_{n=0}^{N-1} x[n] \cos\left(\frac{\pi}{N}nk + \frac{\pi}{2N}k\right) \quad (39)$$

- **Intution behind DCT:** Make the signal behave like an even function. Such as flipping the signal about the y-axis, or reverse the signal and add it to the original signal.
- We can "downgrade" the high frequency components by:
  - **Zeroing out** the high frequency components in the DCT domain. This is done by setting the DCT coefficients above a certain threshold to zero.
  - **Quantization** of the DCT coefficients. This is done by dividing the DCT coefficients by a quantization matrix and rounding the result to the nearest integer. Coarser resolution for high frequency components.

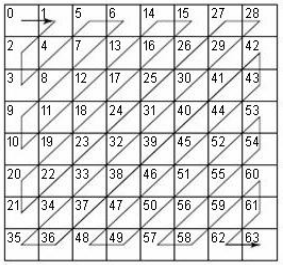
JPEG Compression

Steps:

- **Convert RGB to YCbCr color space**. Y is the luminance (brightness) component, and Cb and Cr are the chrominance (color) components.
- Downsample the chrominance components (Cb and Cr) by a factor of 2. This is done because the human eye is less sensitive to color than brightness.
- **Divide the image into 8x8 blocks.**
- Apply the **DCT to each block**. This transforms the image from the spatial domain to the frequency domain.
  - Each block is transformed by 2D DCT. The result is a set of 8x8 DCT coefficients.
  - The following shows the basis functions used for the 2D DCT



- **Quantize the DCT coefficients** using a quantization matrix. Higher frequency components are quantized more coarsely than lower frequency components.
- Quantization means dividing the DCT coefficients by a quantization matrix and rounding the result to the nearest integer. This reduces the number of bits needed to represent the DCT coefficients.
- Luminance channels are quantized with a different matrix than chrominance channels.
- Apply **zig-zag scanning** to the quantized DCT coefficients. This reorders the coefficients in a way that groups the low frequency components together. (So that RLE and Huffman coding can have a higher compression ratio).



- Apply **run-length encoding (RLE)** to the zig-zag scanned coefficients. This compresses the data by replacing consecutive zeros with a single zero and a count of how many zeros there are. (lossless compression)
- Apply **Huffman coding** to the RLE data. (lossless compression)
- Store the compressed data in a JPEG file format.

Run Length Encoding (RLE)

- RLE is a simple form of lossless data compression. It replaces consecutive repeated values with a single value and a count of how many times it is repeated.
- For example, the sequence [1, 1, 1, 2, 2, 3] would be encoded as [(1, 3), (2, 2), (3, 1)].
- RLE is most effective on data with long runs of repeated values. It is not effective on data with high variability.
- Another RLE technique is to group consecutive zeros together using a (Run, Value) format:
  Run = Number of zeros before a nonzero value.
  Value = The next nonzero value.
  A special End-of-Block (EOB) marker (e.g., (0,0)) is used to signal the end of nonzero values.

Huffman Coding

- Huffman coding is a **lossless** data compression algorithm. It replaces frequently occurring values with shorter codes and less frequently occurring values with longer codes.
- The algorithm builds a **binary tree** based on the frequency of each value in the data. The most frequent values are closer to the root of the tree, and the least frequent values are further away.
- The code is generated by traversing the tree. A left branch is represented by a 0, and a right branch is represented by a 1.
- The resulting binary tree is used to generate **variable-length codes** for each value in the data. All of which are **prefix codes**, meaning no code is a prefix of another code.
- **Algorithm:**
  - Create a min-heap (priority queue) where each node is a character and its frequency.
  - While more than one node in the heap:
    - Renive the two nodes with the lowest frequency.
    - Create a new internal node with these two as children
    - The new nodes frequency is the sum of the frequencies of its children.
    - Insert the new node back into the heap.
  - The remaining node is the root of the Huffman tree.
- With Huffman coding, we need to **send the Huffman tree** along for decoding.

Code	Frequency	Huffman Code
a	14	000
b	1	111010
c	2	11110
d	10	010
e	4	0110
h	3	01110
i	15	10
l	4	1100
o	1	111011
r	3	01111
s	4	1101
t	2	11111
space	11	001
.	2	11100

a:	14 * 3 bits = 42 bits
b:	1 * 6 bits = 6 bits
c:	2 * 5 bits = 10 bits
d:	10 * 3 bits = 30 bits
e:	4 * 4 bits = 16 bits
h:	3 * 5 bits = 15 bits
i:	15 * 2 bits = 30 bits
l:	4 * 4 bits = 16 bits
o:	1 * 6 bits = 6 bits
r:	3 * 5 bits = 15 bits
s:	4 * 4 bits = 16 bits
t:	2 * 5 bits = 10 bits
space:	11 * 3 bits = 33 bits
.:	2 * 5 bits = 10 bits
	Total number of bits = 255 bits

If allocated 8-bits (ASCII) equally to all, we will need 76 characters * 8 bits = 608 bits. Even if you want to "customize" to this string and use only 4 bits, you will need 76 * 4 = 304 bits, which is still larger than 255 bits that Huffman coding gives.

Lempels-Ziv-Welch (LZW) Compression

- LZW is a **lossless** data compression algorithm. It replaces repeated sequences of data with a single code.
- Used in GIF and TIFF image formats, Unix file compression utility compress.
- **Encodes 8 bit data using n bit codes**, where $n > 8$. 0-255 are the original 8 bit data, and 256-4095 are the new codes and data dependent.
- The algorithm starts with a dictionary of all possible single-character sequences. As it processes the data, it adds new sequences to the dictionary.
- When it encounters a sequence that is already in the dictionary, it replaces it with the corresponding code.
- We don't need to send the dictionary along with the compressed data, because the decoder can build the same dictionary from the compressed data.
- **Encoding Algorithm:**
  - Initialize the dictionary with all possible single-character sequences.
  - Read the longest sequence W found in the dictionary.
  - Output its corresponding code.
  - Read the next character K.
  - Add W + K to the dictionary with the next available code.
  - Repeat until the end of the data.
- **Decoding Algorithm:**
  - Initialize the dictionary with all possible single-character sequences.
  - Read the first code and output the corresponding sequence.
  - Set this sequence as the `previous sequence`.
  - **While** there are more codes:
    - Read the next code.
    - **If** the code exists in the dictionary:
      - Let `entry` be the corresponding sequence.
    - **Else** (edge case: code not in dictionary yet):
      - Let `entry` = `previous sequence + first character of previous sequence`.
    - Output `entry`.
    - Add `previous sequence + first character of entry` to the dictionary.
    - Set `previous sequence = entry`.

'ABBAABBAABBAABBAABBAABBAABBAABBAABBA' ← 36 characters = 36*8 bits = 288 bits

Code Book		Output code	
257	'AB'	65	'A'
258	'BB'	66	'B'
259	'BA'	66	'B'
260	'AA'	65	'A'
261	'ABB'	257	'AB'
262	'BAA'	259	'BA'
263	'ABBA'	261	'ABB'
264	'AAB'	260	'AA'
265	'BBA'	258	'BB'
266	'AABB'	264	'AAB'
267	'BAAB'	262	'BAA'
268	'BBAA'	265	'BBA'
269	'ABBAA'	263	'ABBA'
270	'ABBAAB'	269	'ABBAA'
		265	'BBA'

← 15 strings = 15*12 bits = 180 bits

Compression achieved = (288-180)/288*100% = 37.5%

- Because we set it to $4096 = 2^{12}$ entries, we can use 12 bits to represent the codes. Although in this case, 9 bits would have sufficed as 270 codes were used.

Notes to self

- When you perform FFT on a signal $y[n]$, the result Y[k] is complex valued.

$$Y[k] = |Y[k]|e^{j\phi_k} \quad (40)$$

where:
- $|Y[k]|$ is the magnitude
- ϕ_k is the phase of frequency bin k

- When you take the **magnitude of the FFT** result, you are **zeroing out all phase information**. i.e $\phi_k = 0$.
- All sinusoid waves become pure cosines when phase is lost, because cosine is the default phase-zero sinusoid.
- Sine wave: $\sin(2\pi ft) = \cos(2\pi ft - \pi/2)$